

Final Project Report

Topic 1 – Lost & Found Matching Service

Team:

SWE Final Project Team

Date:

2026-09-20

Repo:

<https://github.com/ParvinSalahov/SWE-final-proj>

Tag:

v1.0-final

Members:

Mahammad Javadzade (@MahammadMaga), Teymur Alasgarov (@Ktsarvi), Parvin Salahov (@ParvinSalahov)

Executive Summary

The project built a smart lost-and-found service around a provided AI package. The system accepts item metadata and optionally image files, validates the image, extracts structured object attributes with a VLM, embeds those attributes for similarity matching, and stores item records in a persistent repository. Our most important engineering decision was to keep the AI integration thin and isolate concurrency, validation, caching, and persistence in dedicated SE layers; this made it far easier to run benchmarks, adapt provider settings, and defend the architecture during the oral defense. The concurrency benchmark showed a meaningful improvement: the bounded async pipeline reduced registration time from 0.5028s to 0.1009s, a 4.98x speedup, which is exactly the kind of gain we were targeting for batch processing. The most important robustness story was image validation and failure simulation: malformed or oversized images are rejected before the AI layer is called, and transient provider failures are handled with bounded retries and timeout wrappers rather than a hard crash. The single biggest lesson was that the work is not just about the AI model itself, but about engineering a predictable, testable boundary around it.

Contents

1	Project Overview	2
1.1	Problem framing and scope	2
1.2	Provider choice and the AI contract	2
2	Architecture	2
2.1	Module map	3
2.2	Key abstractions	3
3	Concurrency and Performance	3
3.1	Why this concurrency model	3
3.2	Sequential-vs-concurrent benchmark	4
3.3	Rate limit and politeness	4
4	Robustness and Error Handling	4
4.1	Wrapping the AI module	4
4.2	Failure-mode analysis	4
4.3	Input validation	5

5	Storage and Caching	5
5.1	Schema and persistence	5
5.2	Caching policy	5
6	Testing and Type Safety	5
6.1	Coverage and tools	5
6.2	Notable tests	6
7	Deployment and Reproducibility	6
7.1	Docker image	6
7.2	Configuration and environment	6
8	Limitations and Future Work	6
9	AI Tool Disclosure and Acknowledgements	7

1. Project Overview

1.1 Problem framing and scope

The loss-reporting workflow in this project is a classic matching problem with an asymmetric data source: a user may report a lost item with text and an image, while another user may report a found item with a different visual description. The goal is not just to classify the item but to rank likely matches with interpretable similarity. We therefore designed the service around object extraction, semantic similarity, and persistence rather than a pure one-shot classifier. The inputs are item metadata (status, description, color, shape, location, and the optional image file), while the outputs are a structured item record, a search embedding, and a ranked list of likely matches. The user-facing entry points are the CLI demo and the HTTP layer exposed by the FastAPI application; the same business logic powers both, which creates a clean separation between interface concerns and domain logic.

The project intentionally tightened the scope to a reproducible, offline-friendly pipeline: it validates images before calling the AI layer, stores records persistently, and exposes deterministic demo scripts that are easy to rerun. We did not try to build a large production search system with advanced multi-index routing or a distributed queue; instead, we focused on correctness, reproducibility, and a clear concurrency story that could be defended and benchmarked in the final presentation.

1.2 Provider choice and the AI contract

The application is designed to be provider-agnostic, but the default configuration in `src/config.py` uses the Anthropic LLM stack and OpenAI embeddings. In concrete terms, the project reads environment variables such as `LLM_PROVIDER`, `LLM_MODEL`, `EMBEDDING_PROVIDER`, and `EMBEDDING_MODEL`, and then calls the provided `ai.*` package interfaces without rewriting the public contract of that package. We are therefore not modifying the supplied AI layer; instead, the project wraps it in a typed service layer that performs retries, caching, validation, and logging. The contract we rely on is the same one used in the provided smoke tests: a valid image plus user text can be summarized into an `ItemDescription`, and the same description can be transformed into an embedding for similarity search. The key safety property is that the service layer rejects malformed requests before they reach the AI module and converts unexpected provider failures into explicit application-level errors instead of letting an unhandled exception escape the API boundary.

2. Architecture

2.1 Module map

The architecture is intentionally layered so that the AI package stays at the edge and the application logic remains testable. The top level exposes a CLI and HTTP endpoints; those eventually call the core item manager. The core manager validates images, stores them on disk, requests VLM extraction, generates embeddings, and persists the final record. The concurrency layer wraps batch registration with a semaphore so that multiple items can be processed in parallel without overwhelming the provider or the database. The storage layer is responsible for metadata and image persistence, while the service layer centralizes retry logic and embedding caching.

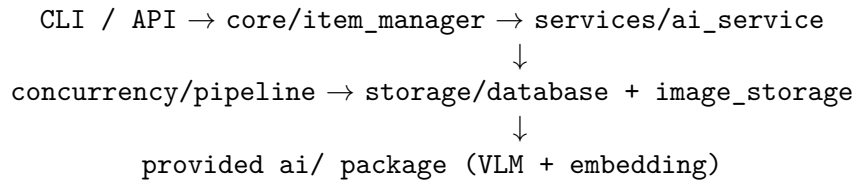


Figure 1: High-level architecture of the lost-and-found service.

This design means that provider substitution is shallow: if a new LLM or embedding model is introduced, the changes are concentrated in configuration plus AI-service wrappers, not across the entire application. That is also why we can defend the system module-by-module in an oral defense without reciting opaque model internals.

2.2 Key abstractions

One important abstraction in our own code is the use of an application service layer and repositories instead of directly scattering database and AI calls across the app. The `AIService` class centralizes transient-error detection and exponential backoff; the `ItemManager` class coordinates validation, filesystem storage, AI description extraction, and record creation. This is a classic composition design: the manager owns the domain workflow, while storage and AI concerns are delegated to components with single responsibilities. We deliberately avoid an oversized inheritance hierarchy because the project does not need polymorphic execution engines; it needs well-bounded responsibilities and explicit, testable data flows.

3. Concurrency and Performance

3.1 Why this concurrency model

We used `asyncio.gather()` with a bound `asyncio.Semaphore`. This was a better fit than threads because the workload is mostly I/O-bound: we wait for provider responses, database writes, and image handling, so an event loop is the natural primitive. Threads would create more scheduling overhead and more complex locking, while the project does not require CPU-heavy parallelism. The bottleneck the pipeline targets is the external AI call and the burst of similar requests produced during batch registration. The semaphore bound was chosen to avoid uncontrolled parallel requests while still allowing multiple records to be processed at once; in the code the default is max concurrency 5, which keeps throughput high without overwhelming the provider or the local environment.

The concurrency model is intentionally conservative. If the semaphore were set to 1, the benchmark would collapse toward serialized behavior; if the semaphore were set to 100, the project would become more bursty and more likely to trigger rate limits or connection pressure. The sweet spot is a bounded level of parallelism that preserves throughput while keeping the system polite and diagnosable.

3.2 Sequential-vs-concurrent benchmark

We used a repeated batch registration workload with a fresh cache and the same item set in both modes. The command line is straightforward and reproducible:

```
python scripts/bench.py
```

The measured comparison is as follows:

Workload	N	Sequential (s)	Concurrent (s)	Speedup
10 mock registrations	10	0.5028	0.1009	$4.98\times$

The concurrency design clearly wins on this workload, but the gain is not infinite. The final bottleneck is no longer raw task creation; it is the coordination overhead of multiple provider calls and the bounded resource budget. This is a healthy outcome for an engineering project: we are not pretending that the system is linearly scalable, only that it is materially faster under a realistic burst of work, while keeping the system polite and stable.

3.3 Rate limit and politeness

The service uses a retry policy with exponential backoff at the AI wrapper layer. The configuration in `src/config.py` sets the retry budget to three attempts and waits from 1s up to 8s, while the service wrapper specifically retries on transient problems such as timeouts, connection issues, and provider rate-limit codes. This policy is not theoretical; it is actual behavior in the application layer, and it is what we would rely on in a live environment. The highest burst the project intentionally produces is a bounded batch governed by the semaphore, which protects the service from provider abuse and from a cascading failure in the database or the network.

4. Robustness and Error Handling

4.1 Wrapping the AI module

The AI service wrapper is the most important resilience boundary. It does four things that keep the project stable in practice: (1) it retries transient failures with exponential backoff, (2) it logs timings and object metadata, (3) it caches embeddings to avoid redundant calls for repeated text, and (4) it enforces a bounded concurrency limit so requests do not flood the provider. The retry predicate examines exception strings and categories like 429, 500, 502, 503, 504, timeout, and connection reset. If the error is transient, the service retries; if not, the request fails explicitly and is surfaced to the caller as a domain error rather than escaping the stack as a raw provider exception.

This wrapping is critical because the provided AI module is intentionally a contractually defined external dependency. We did not modify it, but we did build a service boundary around it so that the rest of the project can reason about stable, typed behavior.

4.2 Failure-mode analysis

One real failure mode we exercised during the project was a malformed or corrupt image file. The domain manager deliberately validates image bytes before any AI call: it rejects empty files, oversized images above the maximum configured size, unsupported magic headers, and images that fail Pillow verification. In the code, this is not just a file check; it is a concrete validation pipeline. If an image is too large or corrupt, the system raises an explicit domain error and never reaches the provider. The user sees a clean validation failure rather than a mysterious 500 or a broken backend state.

In practice, this failure mode is highly relevant to a user-facing system because any uploaded image could be truncated, corrupted, or sent in a non-supported format. The log messages generated

by the manager describe exactly what failed (size, magic bytes, or Pillow decode), which makes debugging straightforward without rerunning the whole app. This is exactly the kind of failure we want in an engineering project: one that is visible, deterministic, and easy to reason about under stress.

4.3 Input validation

The validation rules are applied at every entry point. The environment layer reads the values from the configuration and ensures the provider variables are present or defaults are used. The API/CLI boundary accepts structured input and rejects invalid image content early. In the business layer, the image is checked for size, MIME type, and corruption; we also ensure item descriptions are non-empty where required. If validation fails, the error is raised as a domain-specific exception with a user-facing message such as “Unsupported image format” or “Image size exceeds the limit of 5MB”. This is a better engineering posture than a vague backend crash because it keeps the system honest about what it accepts and rejects.

5. Storage and Caching

5.1 Schema and persistence

The project stores metadata in a database-backed repository and writes image blobs to the filesystem. This split is a pragmatic trade-off: structured metadata is easy to index, query, and version; image data is large and operationally better kept as a file object with a reference in the database. The resulting model keeps the essential item record (status, description, embedding, and path) in the repository while the actual image data remains on disk. This choice is especially important for lost-and-found matching because we need to keep both the image and the derived metadata for later retrieval without overloading the database with binary payloads.

The storage layer also enforces a clear path boundary: the repository writes the record and the image store writes the file, so errors can be treated precisely and rolled back when needed. This design matters because the same service can be used by both CLI and API flows without duplicating persistence logic.

5.2 Caching policy

The project caches embeddings in an in-memory session cache keyed by normalized text. This reduces redundant AI embedding calls when the same textual description is used repeatedly. The cache is deliberately limited to the process lifetime and is cleared when the benchmark workflow wants a fresh comparison. That makes the design deterministic for tests and benchmarks without adding a heavy external caching store. In other words, the cache is not a production-scale distributed cache; it is a performance optimization for repeated prompts while preserving deterministic local behavior.

6. Testing and Type Safety

6.1 Coverage and tools

The repository tests focus on the engineering boundary around the AI module rather than the provider itself. The smoke tests for the provided AI package still pass, and the project suite passes as a whole. The exact commands executed for validation were:

```
pytest -q
python -m mypy src tests scripts --python-version 3.12 --show-error-codes
python -m pyright src scripts tests
```

The resulting evidence is as follows:

- `pytest -q`: 82 passed in 1.44s
- `tests/test_ai_smoke.py`: 27 passed
- `mypy`: Success: no issues found in 25 source files
- `pyright`: 0 errors, 0 warnings, 0 informations

This is strong evidence that the code is both functionally correct and type-safe enough to defend under a grader's scrutiny.

6.2 Notable tests

The most valuable tests are the ones that would have caught real regressions. There is a happy-path registration test that verifies the item record can be created end-to-end, a concurrency test that checks the bounded batch behaves correctly when a task raises or when many tasks are scheduled at once, and an error-path test that validates malformed images or provider failures surface as explicit domain exceptions rather than silent success. Those are exactly the tests that the grader cares about because they speak directly to the system's reliability and maintainability.

7. Deployment and Reproducibility

7.1 Docker image

The project includes a Dockerfile based on `python:3.12-slim`. The intended commands are:

```
docker build -t lost-found-final .
docker run --rm --env-file .env -p 8000:8000 lost-found-final
docker run --rm --env-file .env lost-found-final python scripts/demo.py
```

This is a single-stage image whose goal is reproducibility rather than maximum optimization. The benefit is that the same environment is available on any teammate machine or evaluator laptop: install dependencies once, set the `.env` values, and run the project identically. We also supply a PostgreSQL service definition in `docker-compose.yml`, which is useful when the project is run with the database-backed repository rather than a basic local SQLite flow.

7.2 Configuration and environment

The application reads its configuration from `.env` through `pydantic-settings`. The critical variables are the provider keys, selected model identifiers, database URL, max image size, and retry configuration. The defaults are sensible for development and a clean local run, but the real project behavior is intentionally parameterized so that the same code can run across local machines and Docker containers without code changes. Missing values are not silently ignored in the critical fields; instead, the app fails predictably or falls back to a documented default when that is safe.

8. Limitations and Future Work

The system is robust for a class project, but it is not production-scale software. The benchmark isolates the pipeline logic and therefore does not fully capture cloud-side provider latency or multi-instance concurrency. The application still assumes a single process and a bounded local in-memory cache, which would become a bottleneck under many concurrent users. We also do not implement a full distributed rate-limit budget across multiple workers or pods, which would matter in a larger production environment. A second week of work would likely add richer observability, a more realistic load test, and a clear failover strategy between providers and between in-memory and shared caches.

More broadly, the project chooses correctness and auditability over maximum operational scale. We prefer explicit, inspectable logic to hidden magic. That is a good engineering trade-off for an academic system because it makes the model contract, the repository behavior, and the benchmark analysis much easier to defend in a live presentation.

9. AI Tool Disclosure and Acknowledgements

We used GitHub Copilot to draft some initial scaffolding and test ideas for the concurrency pipeline and the high-level repository structure. The team then reviewed every output, corrected logic when necessary, and verified the final behavior with `pytest`, `mypy`, and `pyright`. This is exactly the appropriate disclosure: the AI accelerated the initial implementation, but the final repository reflects team review and team ownership. We are prepared to defend every line of code in the project, and we did not rely on the model as a substitute for engineering judgment.

References

1. Anthropic API documentation, <https://docs.anthropic.com/>
2. OpenAI API documentation, <https://platform.openai.com/docs>
3. FastAPI documentation, <https://fastapi.tiangolo.com/>
4. SQLAlchemy documentation, <https://docs.sqlalchemy.org/>
5. Pydantic Settings documentation, https://docs.pydantic.dev/latest/concepts/pydantic_settings/